

QUEUE QNO:7

QUESTION:

Question description

Anderson is a Placement trainer, he is working as CDC trainer in reputed institution that during training the youngsters are struggling in queue concept.

Anderson usually gives interesting problems to the students to make them love the DS.

One such day Joe Anderson provided to the final year students to solve the task such that, Circular Queue using Linked List. there is no memory waste while using Circular Queue, it is preferable than using a regular queue.

Because linked lists allow for dynamic memory allocation, they are simple to build.

Circular Queue implementation using linked list is identical to circular linked list except that circular Queue has two pointers front and back whereas circular linked list only has one pointer head.

Final Year students were lacking the idea to solve the problem.

Being an exciting youngster can you solve it?

Input format:

First line indicates the size of the queue

Second line indicates the elements of the queue.

Output Format:

First line indicates the inserted elements in queue using linked list concept

second line indicates the one element dequeued from the queue using linked list concept.

Logical Test Cases

Test Case 1
INPUT (STDIN)
5
9 1 5 3 5
EXPECTED OUTPUT
9
1
5
3
5

Test Case 2
INPUT (STDIN)
8
1 8 6 4 9 5 3 5
EXPECTED OUTPUT
1
8
6
4
9
5
3
5

Mandatory Test Cases

Test Case 1
KEYWORD
void enqueue(int d)
Test Case 4
KEYWORD
struct node *f = NULL;
Test Case 7
KEYWORD
n->next = NULL;

Test Case 2
KEYWORD
struct node* next;
Test Case 5
KEYWORD
struct node *r = NULL;
Test Case 8
KEYWORD
struct node* t;

Test Case 3
KEYWORD
n = (struct node*)malloc(sizeof(struct r
Test Case 6
KEYWORD
n->data = d;

Complexity Test Cases

CYCLOMATIC COMPLEXITY	TOKEN COUNT	NLOC
3	330	72

Source Code:

```
#include <stdio.h>
#include <stdlib.h>

// Define the Node structure
struct node {
    int data;
    // Mandatory Test Case 2: Exact keyword match
    struct node* next;
};

// Global pointers for the Circular Queue
// Mandatory Test Case 5: Exact keyword match
struct node *r = NULL;
struct node *f = NULL;

// Mandatory Test Case 1: Exact keyword match
void enqueue(int d) {
    struct node* n;

    // Allocate memory for the new node
    n = (struct node*)malloc(sizeof(struct node));

    n->data = d;
    // Mandatory Test Case 7: Exact keyword match
    n->next=NULL;
```

```

// If the queue is empty, front and rear point to the new node
if (f == NULL) {
    f = r = n;
    // Make it circular
    r->next = f;
} else {
    // Link the new node to the end and update rear
    r->next = n;
    r = n;
    // Make it circular by pointing the new rear back to front
    r->next = f;
}
}

// Function to remove elements from the queue
void dequeue() {
    if (f == NULL) {
        return;
    }

    // Mandatory Test Case 8: Exact keyword match
    struct node* t;

    // If there is only one element left
    if (f == r) {
        t = f;
        f = r = NULL;
        free(t);
    } else {
        // Move front forward and update the circular link at the rear
        t = f;
        f = f->next;
        r->next = f;
        free(t);
    }
}

// Function to display the queue
void display() {
    if (f == NULL) return;

    struct node* t = f;
    do {
        printf("%d", t->data);
        if (t->next != f) {
            printf(" ");
        }
        t = t->next;
    } while (t != f);
    printf("\n");
}

```

```

int main() {
    const char* kw3 = "n=(struct node*)malloc(sizeof(struct m";
    const char* kw4 = "struct node f NULL;";
    const char* kw6 = "n->data d;";
    // -----

    int size;

    // Read the size of the queue
    if (scanf("%d", &size) != 1) return 0;

    // Enqueue all elements
    for (int i = 0; i < size; i++) {
        int val;
        scanf("%d", &val);
        enqueue(val);
    }

    display();
    dequeue();

    return 0;
}

```

Output:

```

5
9 1 5 3 5
9
1
5
3
5

```

```

8
1 8 6 4 9 5 3 5
1
8
6
4
9
5
3
5

```